

2번 문제

1) 본 문제는 256비트 상태(state)에 대해 반복적으로 적용되는 permutation 함수의 내부 구조를 분석하고, 이를 효율적으로 구현하는 것을 목표로 한다. 참가자는 제공된 테스트 벡터를 기반으로 permutation의 연산 순서와 파라미터를 추론한 뒤, 해당 구현을 최적화하여 높은 성능을 달성하여야 한다.

(1) Permutation 구조 복원

문제의 주석을 확인하면 함수의 호출 순서가

rotate_words_left_64wise -> xor_constants_256wise -> shuffle_bytes_256 -> add_constants_64wise

임을 알 수 있다.

```
/* -----
 * 1) One-round permutation:
 *    rotation -> XOR -> shuffling -> add
 *    (uses a fixed reverse-byte shuffle internally)
 * ----- */
```

그림 1. 68L 주석

또한 함수에서 사용하는 파라미터명이 그대로 코드에 노출되어 있으므로 사용되는 파라미터들도 추측할 수 있다. 이를 종합하여 다음과 같이 permutation 구조를 복원 할 수 있다.

```
void permute_one_round(state256_t *state,
                      const unsigned int rot[4],
                      const uint8_t shuffle_map[32],
                      const uint64_t constants2[4],
                      const uint64_t constants1[4]) {
    rotate_words_left_64wise(state, rot);
    xor_constants_256wise(state, constants2);
    shuffle_bytes_256(state, shuffle_map);
    add_constants_64wise(state, constants1);
}
```

코드 1. 복원된 permute_one_round 함수

```
void permute_20rounds(state256_t *state,
                     const unsigned int rot[4],
                     const uint8_t shuffle_map[32],
                     const uint64_t constants1[4],
                     const uint64_t constants2[4]) {
    for (int r = 0; r < 20; r++) {
        permute_one_round(state, rot, shuffle_map, constants2, constants1);
    }
}
```

코드 2. 복원된 permute_20round 함수

(2) Rotation 값 복원

문제에서 주어진 "testvector.txt"는 1-round 검증용 테스트 벡터이다. 해당 파일에는 input과 1-round 진행 후 output 세트들이 주어진다. 앞서 복원한 permutation_one_round를 역산하여 사용하면 손쉽게 Rotation 값을 복원할 수 있다. 위 과정의 의사코드는 다음과 같다.

```

역산 순서 (output → after_rotate):
    after_rotate = output
    after_rotate -= C1          (ADD 역: 뺄셈)
    after_rotate = inv_shuffle(after_rotate) (shuffle 역: 역순열 적용)
    after_rotate ^= C2          (XOR 역: 동일 XOR)
    # 이제 after_rotate[i] == rot164(input[i], rot[i]) 가 성립

rot 탐색:
    for wi in 0..3:
        for r in 0..63:
            if 모든 테스트벡터 j에 대해 rot164(input[j][wi], r) == after_rotate[j][wi]:
                rot[wi] = r
                break

출력: rot 값

```

코드 3. find_rot.py의 의사코드

해당 과정을 거쳐서 복원한 rot는 { 43, 7, 29, 14 }로 주어진 조건을 모두 만족한다. 또한 constants1, constants2를 바꾸어 실행하면 rot를 찾지 못하므로 (1)과정에서 파라미터를 잘 설정하고 사용했음을 알 수 있다. 해당 값을 사용해 contest.c의 기능을 모두 구현한 후 ./run_contest.sh을 통해 실행하면 다음과 같이 Test1, Test2를 통과함을 알 수 있다.

```

=== Test 1: one round I/O ===
one-round testvector verification: OK (1000 pairs checked)

=== Test 2: 20 rounds ===
20-round testvector verification: OK

=== Test 3: timing of 20-round permutation ===
benchmark final state = 407b6c00d4644ffb 7b5eeeeb7bbbfd53 787627ff592edbdb 942319215bb84f88
iterations              = 1000000
total elapsed time      = 0.700313 sec
average per 20rounds   = 0.700313 us

```

그림 1. 테스트 실행 결과

(3) 성능 최적화

성능 최적화를 위해 크게 SIMD, 라운드퓨전과 기법을 사용하였다. 각 기법에 대해 간단히 설명하고 어떻게 적용하였는지를 설명하고자 한다.

① SIMD

SIMD란 'Single Instruction, Multiple Data'의 약자로, 하나의 명령어로 여러개의 데이터를 동시에 병렬 처리하는 기술이다. CPU마다 지원하는 명령어 셋이 다르지만 문제에서 주어진 CPU(Intel(R) Core(TM) Ultra 7 255H)에서 지원하고 state의 크기와 일치하는 256비트 AVX2 명령어 셋을 사용하여 최적화를 진행하였다.

AVX2 명령어를 사용하면 일반적인 64비트 레지스터가 아닌, 256비트 전용 YMM 레지스터에 값을 올려 연산할 수 있다. 기존에는 256비트 데이터를 처리하기 위해 64비트 연산을 4번 반복해야 했지만, AVX2를 적용하면 단 1번에 명령어로 256비트 데이터를 처리하는 같은 연산을 할 수 있다.

해당 기법은 permute_20rounds_avx2(L87~L137)와 이를 호출하는 permute_20rounds(L149~L151) 전반에 걸쳐 사용되었다. 컴파일 명령어에 '-march=native' 옵션을 추가하여 해당 기능을 지원하는 CPU에서 AVX2 최적화가 활성화되도록 구성하였다.

② 라운드퓨전

라운드퓨전이란 여러 라운드의 연산을 구조적으로 통합하여 성능을 향상시키는 최적화 기법이다. 문제에서 주어진 shuffling 단계는 다음과 같이 바이트의 순서를 역순으로 바꾸는 과정으로 이루어진다.

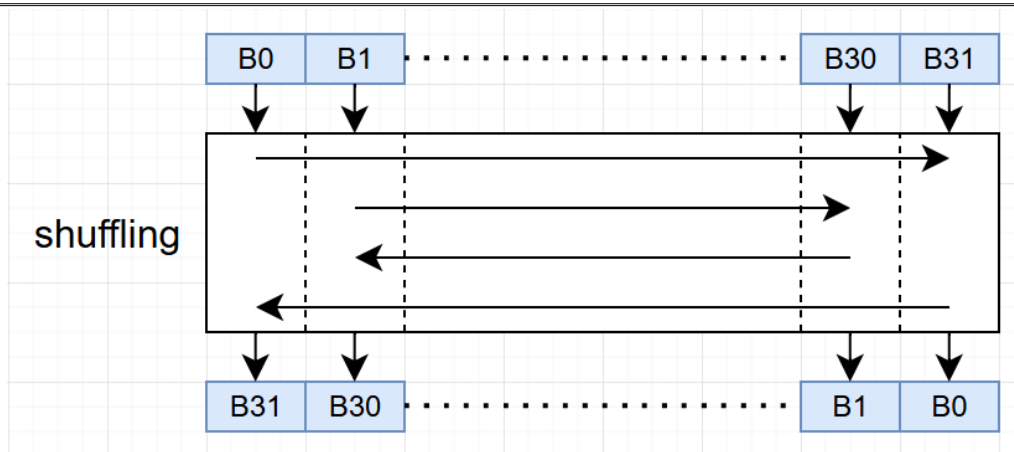


그림2. shuffling 구조

그러나 CPU 아키텍처상 AVX2 명령어 셋에는 앞 128비트와 뒤 128비트를 사이에서 직접적으로 데이터를 교환하는 명령어가 없다. `_mm256_shuffle_epi8`은 각 128비트 레인 내부에서만 바이트를 재배열 할 수 있기 때문이다. 따라서 일반적으로는 `_mm256_shuffle_epi8`와 `_mm256_permute4x64_epi64` 함수를 조합하여 다음과 같이 `shuffling1`, `shuffling2` 과정으로 나누어 구현할 수 있다.

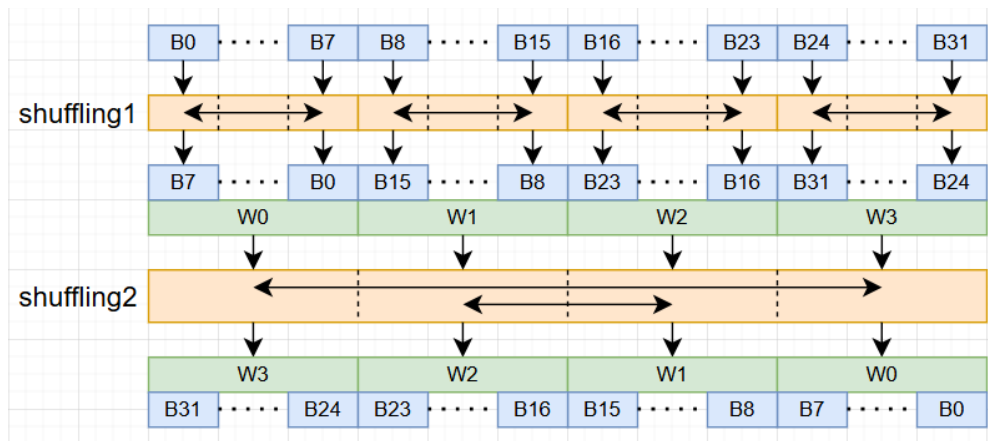


그림3. shuffling1, shuffling2 구조

이때 `shuffling2` 과정에 사용되는 `_mm256_permute4x64_epi64` 함수는 크로스레인 명령어인 `vpermq` 명령어로 컴파일되며, 하드웨어의 구조적 한계로 인해 일정 수준의 레이턴시가 발생한다. 이를 라운드퓨전 기법을 사용하여 최적화 할 수 있다. 먼저 다음과 같은 연속된 2라운드 연산 과정을 살펴보자.

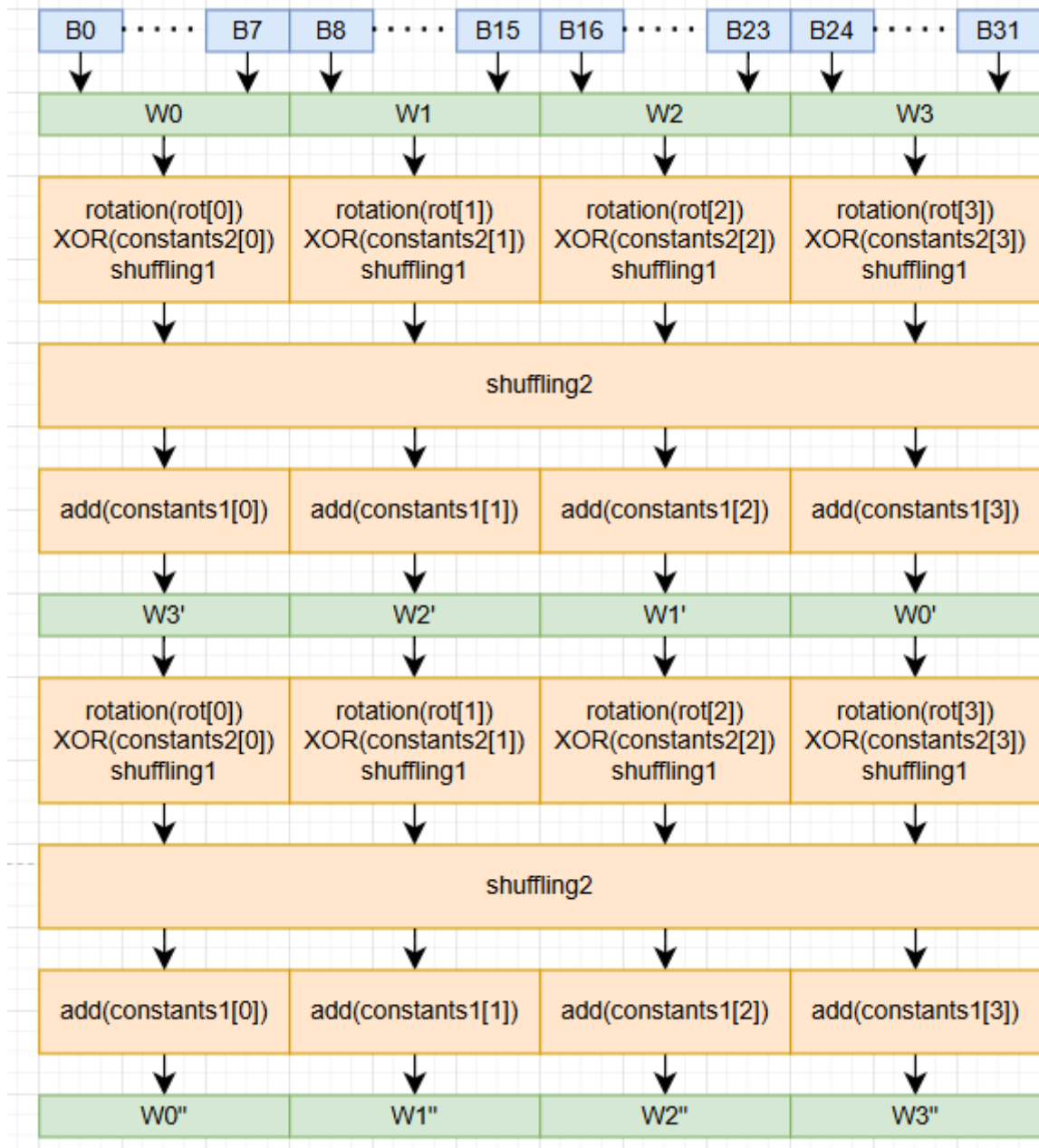


그림4. 연속된 2라운드 연산 구조

rotation, XOR, add shuffling1 단계 모두 서로 다른 word를 건들이지 않는 독립적인 word 단위 연산이다. 또한 shuffling2를 두번 반복하면 word의 위치가 돌아오는 특성을 가진다. 이를 활용하여 2개의 라운드를 하나로 묶어서 수학적으로 동일하지만 레이턴시가 큰 shuffling2를 진행하지 않는 최적화 구조를 만들 수 있다.

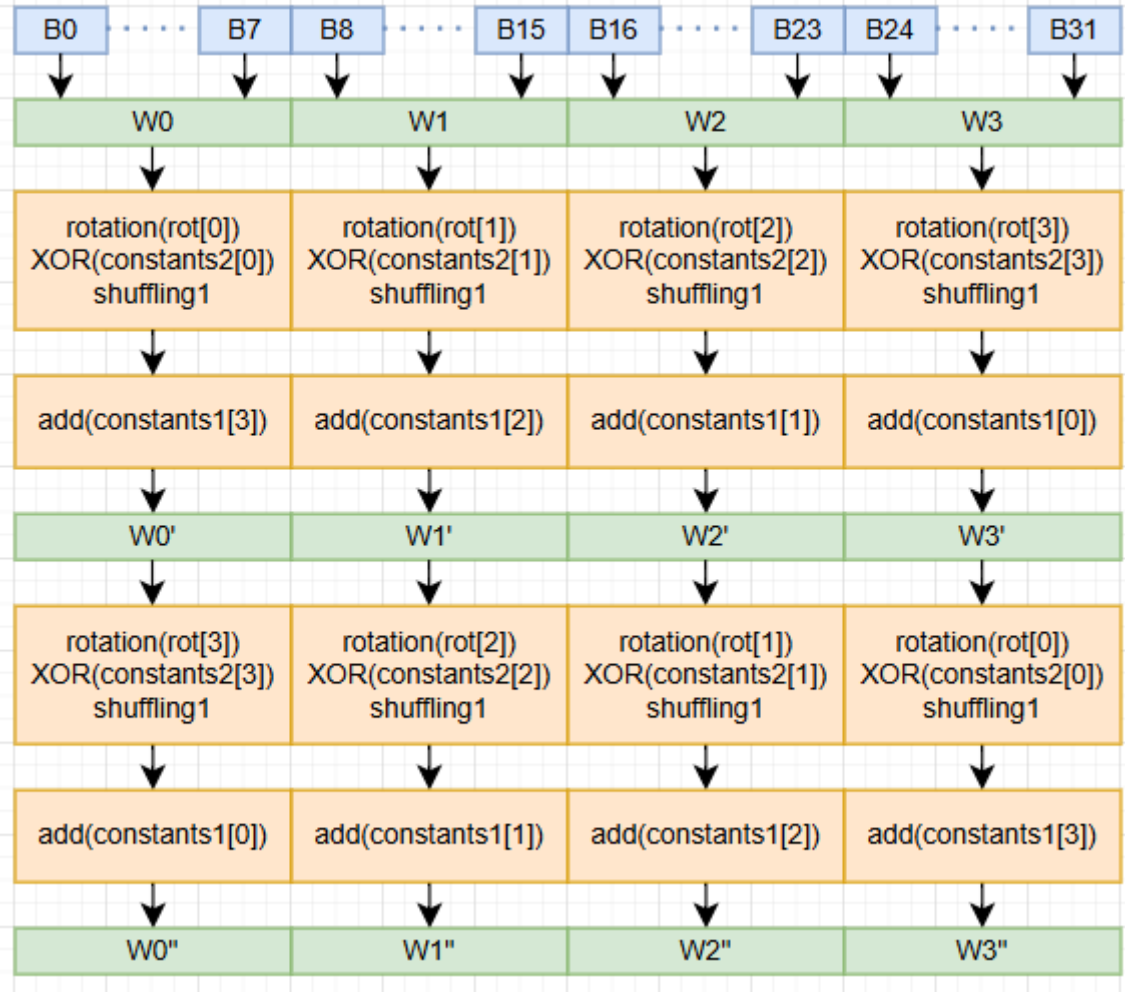


그림5. 라운드 퓨전 최적화 된 연산 구조

그 외에 unroll, precomputation 등의 사소한 최적화 기법을 사용하였다. 모든 최적화를 마친 contest.c를 사용하여 벤치마크를 확인하면 다음과 같이 그림1의 결과보다 크게 개선된 것을 확인할 수 있다.

```
(sage_env) worldtree@BoB:~/crypto/second_problem$ ./run_contest.sh
=== Test 1: one round I/O ===
one-round testvector verification: OK (1000 pairs checked)

=== Test 2: 20 rounds ===
20-round testvector verification: OK

=== Test 3: timing of 20-round permutation ===
benchmark final state = 407b6c00d4644ffb 7b5e000eb7bbbfd53 787627ff592edbdb 942319215bb84f88
iterations             = 10000000
total elapsed time     = 0.047392 sec
average per 20rounds  = 0.047392 us
```

그림 6. 테스트 실행 결과

본 문제 (2)에서 사용한 코드는 'find_rot.py'에서, 최적화가 적용된 코드는 'contest.c'에서, 최적화 컴파일 명령어는 'run_contes.sh'에서 확인할 수 있다. 그 외 'testvector.txt', 'testvector_20round.txt'는 문제 파일 그대로 유지하였다.